# Qiskit Fall Fest 2026 Hackathon - Basic Track

## Overview

Teams create a verified Bell pair between two fixed qubits ("ports") on three different processor models, then test whether one error-detection strategy improves it and what it costs. The challenge is "Geometry-aware quantum cloud challenge" by Steven Rayan (quanTA, University of Saskatchewan), provided by IBM Quantum for Qiskit Fall Fest 2026.

**Who this track is for.** For participants getting started with quantum computing and Qiskit. Coupling maps, transpiler routing, noise models and Pauli expectation values are explained in this document, and the Explorer level described in the Challenge tasks section is the minimum.

**Common definitions.** The three coupling graphs, the two port qubits on each, the basis gates, the noise model with its parameters and the shot count are provided by the organizers. Every team must use them unchanged, so that results can be compared across teams.

## Background and problem statement

**Problem statement.** Create a verified Bell correlation between two designated ports on each of three processor models. Establish an ideal and a noisy baseline, then test one justified error-detection or protection strategy. Compare the resulting fidelity with the circuit cost and any discarded shots, and explain the differences between the three architectures.

**Connectivity, routing and noise.** On real hardware a two-qubit gate can only act on qubit pairs that are physically connected; the connections form the *coupling graph*. When a circuit needs two unconnected qubits to interact, the transpiler inserts SWAP gates to move the states next to each other (routing). Each SWAP is three CNOTs, so under noise these extra operations lower the quality of the result. The two designated ports in this challenge are fixed nodes on the graph, and they are generally not neighbours.

**Research context (quanTA).**

- *Geometry and error protection.* quanTA's research connects circuit diagrams and quantum error correction with the geometry of physical implementations.
- *Hyperbolic superconducting architectures.* Finite hyperbolic lattices suggest alternative patterns of connections and cycles. This challenge tests what those patterns mean for routing and error-detection circuits.
- *A decade of cloud access.* IBM opened its five-qubit cloud processor in 2016. Qiskit now provides a common framework for comparing historical, current and hypothetical processor models.

**The three processor models.**

| Model | Coupling graph | Noise and gates | Role in the comparison |
|---|---|---|---|
| 2016-inspired | Five qubits, five-node | Synthetic noise model | Historical baseline |

| Model | Coupling graph | Noise and gates | Role in the comparison |
|---|---|---|---|
| reference | reference graph | | |
| Contemporary IBM-style | Backend snapshot or fake backend | Native target and gate set of that backend | Current compilation baseline |
| Hyperbolic-inspired | Coupling graph derived from a finite hyperbolic lattice | Circuit-level model only (hypothetical hardware) | Test of an alternative geometry |

In current Qiskit, a contemporary model can be loaded from `qiskit_ibm_runtime.fake_provider` (for example a `FakeXxx` backend), and the other two can be built as a `CouplingMap` with an Aer `NoiseModel`. The exact graphs, ports and noise parameters are the common definitions announced by the organizers.

## Challenge tasks

The work is a 3 × 3 grid: three processor models, each run ideal, noisy, and noisy with protection. All seven tasks are required for every team; the level of ambition chosen in Task 5 (Explorer, Builder or Research) sets how far a team goes.

### Task 1: Set up the three processor models

Load or build the three coupling graphs from the common definitions and mark the two designated ports on each. Draw each graph with the ports highlighted; these drawings are a required deliverable.

### Task 2: Prepare the Bell state and compile it

The target is $|\Phi+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$ on the two ports. The logical circuit is a Hadamard on the first qubit followed by a CNOT from the first to the second.

Transpile this circuit for each model with the two logical qubits placed on the ports (for example through `initial_layout`). Record, for every transpiled circuit: depth, number of two-qubit gates, number of SWAPs inserted, and physical qubits used. If the ports are not neighbours, routing will add SWAPs; this is where the geometry enters.

### Task 3: Verify the Bell correlation

Measuring only in the Z basis is not enough. A classical mixture of 00 and 11 gives the same perfectly correlated Z results as the Bell state. Verification therefore uses three correlations, XX, YY and ZZ.

**Fidelity formula.** The Bell-state projector can be written in Paulis. $|\Phi+\rangle$ is the +1 eigenstate of both XX and ZZ, and XX·ZZ = −YY, so

$$|\Phi^+\rangle\langle\Phi^+| = \tfrac{1}{2}(II + XX) \cdot \tfrac{1}{2}(II + ZZ) = \tfrac{1}{4}(II + XX - YY + ZZ)$$

Taking the expectation value in the measured state ρ gives the fidelity used in this challenge:

$$F(\Phi^+) = \mathrm{Tr}(\rho\,|\Phi^+\rangle\langle\Phi^+|) = [1 + \langle XX\rangle - \langle YY\rangle + \langle ZZ\rangle] / 4$$

**Worked check.**

| State | ⟨XX⟩ | ⟨YY⟩ | ⟨ZZ⟩ | F |
|---|---|---|---|---|
| Ideal $\|\Phi+\rangle$ | +1 | −1 | +1 | 1 |
| Classical mixture ½($\|00\rangle\langle 00\|$ + $\|11\rangle\langle 11\|$) | 0 | 0 | +1 | 0.5 |

F above 0.5 rules out any unentangled state. Because F is estimated from finite shots, a Bell correlation counts as verified only if $F - 2\sigma_F > 0.5$ (about 95% confidence), with $\sigma_F$ as defined below.

**How to measure each term.** Either use an Estimator primitive with the observables XX, YY, ZZ (as `SparsePauliOp`), or run three circuits with a Sampler: measure directly for ZZ; apply H to both ports before measuring for XX; apply $S^\dagger$ then H to both ports for YY. From N shots, $\langle PP\rangle = (N_{even} - N_{odd})/N$, where even/odd is the parity of the two-bit outcome.

**Uncertainty.** For N independent shots per setting, the statistical error of each correlator is $\sigma_{PP} = \sqrt{(1 - \langle PP\rangle^2)/N}$. Since the three settings are separate circuits,

**$\sigma_F = ¼ \sqrt{\sigma_{XX}^2 + \sigma_{YY}^2 + \sigma_{ZZ}^2}$**

Bootstrap resampling of the counts is an acceptable alternative.

### Task 4: Fill the core comparison

| Processor model | Ideal baseline | Noisy baseline | Noisy + protection |
|---|---|---|---|
| 2016-inspired | Case 1 | Case 2 | Case 3 |
| Contemporary IBM-style | Case 4 | Case 5 | Case 6 |
| Hyperbolic-inspired | Case 7 | Case 8 | Case 9 |

For each model, compare the noisy baseline with the protected result and its extra cost.

**Required measurements for every case:**

| Measure | Report |
|---|---|
| Output quality | Bell fidelity with uncertainty |
| Circuit cost | Depth and number of two-qubit gates |
| Routing | SWAP overhead |
| Qubit use | Physical qubits, including ancillas |
| Post-selection | Acceptance fraction and number of discarded shots |

### Task 5: Add one justified protection strategy

Strategies to choose from:

- **Parity or stabilizer checks.** Extra measurements on ancilla qubits reveal particular errors without measuring the encoded information directly.
- **Syndrome post-selection.** Keep only the shots that pass a specified check. Any gain in conditional fidelity must be reported together with the fraction of accepted shots.
- **Geometry-aware encodings.** More advanced teams can adapt an encoding or check network to the available connections and measure its overhead.

**Example for Explorer level: a ZZ parity check.** $|\Phi+\rangle$ has ZZ = +1. Add an ancilla in $|0\rangle$, apply CNOT from port 1 to the ancilla and CNOT from port 2 to the ancilla, then measure the ancilla. Outcome 0 means ZZ = +1; keep only those shots.

- ZZ commutes with XX and YY, so this check does not disturb the verification measurements.
- It detects X or Y errors on one port (they flip ZZ). It does not detect Z errors; a second ancilla checking XX does (prepare it in $|+\rangle$, CNOT from the ancilla to each port, H, measure).
- The ancilla must interact with both ports. On a sparse graph that needs extra SWAPs, and errors in the check itself can reject good shots. Whether the check pays for itself depends on the geometry, which is the point of the comparison.

**Levels of ambition.**

| Level | Suggested approach | Main comparison |
|---|---|---|
| Explorer | Bell preparation and a simple error-detection check | Ideal, noisy and protected results |
| Builder | Placement, routing or improved checks | Fidelity benefit versus extra operations |
| Research | Geometry-aware encoding or stabilizer network | Scaling and sensitivity to noise |

For the Basic track, Explorer is the minimum. Builder-level work scores higher under "Quality of architecture comparison" and "Protection strategy". Research-level teams may want to consider the Advanced track instead.

### Task 6: Interpret the results

- **Noise and compilation.** Relate each change in fidelity to the operations added by compilation and to the assumed noise model.
- **Protection overhead.** Assess whether the protection method makes up for its extra gates, ancillas and discarded shots.
- **Architectural conclusions.** Separate the effect of connectivity from differences in gate sets or noise. A result from one finite model does not establish a universal advantage.

One way to isolate connectivity: rerun all three graphs with the same noise model and gate set, so the only remaining difference is the graph.

### Task 7: Recommend one design choice

Propose one specific hardware or software design choice for future cloud quantum systems, supported by your results.

### Challenge rules

| Common requirements (fixed for all teams) | Choices and extensions (team decides) |
|---|---|
| Use the common processor definitions, fixed output ports and noise assumptions. | Choose the layout, routing method and protection strategy. |
| Complete the three-model comparison and test one justified protection method. | Explore alternative noise models, larger circuits or geometry-aware encodings. |
| Report reproducible results, including added operations and post-selection yield. | Keep optional studies and QPU results separate from the common benchmark. |

## Submission

Upload everything to a single **public** GitHub repository, then submit **only the repository link**. The repository must contain:

1. **Code**: complete, runnable source code
2. **Recorded presentation**: a video walkthrough of your approach and results
3. **Report**: a written report describing the problem, method and findings

Rules, the code of conduct and all other details are on the hackathon website. For questions, write to qff@iitj.ac.in with "Hackathon" in the subject line.

## Judging criteria

Half the score (30% + 20%) rests on correct, reproducible numbers and a sound comparison of the three architectures.

| Criterion | Weight | What scores high |
|---|---|---|
| Scientific correctness and reproducibility | 30% | Fidelity computed from XX, YY and ZZ with uncertainties; notebook reruns to the same numbers; common definitions used unchanged. |
| Quality of architecture comparison | 20% | All nine cases filled; differences traced to SWAPs, gate count and noise; connectivity separated from gate set and noise. |

| Criterion | Weight | What scores high |
|---|---|---|
| Protection or error-detection strategy | 20% | Strategy is justified for the errors it targets; conditional fidelity always shown with acceptance fraction; overhead counted honestly. |
| Effective use of Qiskit | 15% | Transpiler options, primitives, fake backends and Aer noise models used as intended with current APIs. |
| Evidence for the future-system recommendation | 10% | Recommendation follows directly from the measured numbers and states its limits. |
| Clarity of presentation | 5% | Graphs, circuits and the results table are readable; claims are stated plainly. |

**Scoring scale.** Each judge scores each criterion from 0 to 10; the weighted sum is scaled to 100 (score × weight ÷ 10). The final score is the mean across judges.

## Resources

Starting points:

- IBM Quantum documentation: transpilation, primitives (Estimator, Sampler) and fake backends
- Qiskit Aer documentation: building noise models for the 2016-inspired and hyperbolic-inspired graphs
- Foundations of quantum error correction (IBM Quantum Learning course): stabilizers and parity checks, for Builder and Research levels
- Error mitigation and suppression techniques: background only; mitigation is not required in this track

**Original prompt:** "Geometry-aware quantum cloud challenge: Hackathon overview", Steven Rayan, quanTA, University of Saskatchewan.